

Reviewer Pack — Minimal Order of Eta-Invariants and Circuit Monotone Width I...

Entry 63ead587425e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 16:18:48 UTC

Minimal Order of Eta-Invariants and Circuit Monotone Width Inequality

Entry ID: 63ead587425e **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 16:18:48 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Eta Invariant) **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

For every Boolean circuit C with n inputs, the minimal order of the eta-invariant $\eta(C)$ associated with its term-overlap graph $G(T_C)$ is bounded by the circuit's monotone width $w_m(C)$, such that $\eta(G(T_C)) = \Theta(w_m(C))$.

Rationale (proposer's reasoning):

The eta-invariant has been historically used in algebraic topology to study the structure of spaces, which could potentially reveal deep connections between topological properties and algorithmic efficiency. The monotone width of a circuit captures its computational difficulty, and a strong bound would suggest that eta-invariants can serve as an effective complexity measure.

Taxonomy category: EtaInvariant (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 44453f919fe51b8b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between $\eta(G(T_C))$ and $w_m(C)$ exceeds 0.8 for all circuits with $n \leq 40$, across 30 random seeds. It is falsified if this condition is not met.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "eta-invariant" AND "circuit monotone width" - "term-overlap graph" AND "Boolean circuit complexity" - "Minimal order of eta-invariants" AND "monotone width inequality"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.4s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n):
        circuit = []
        for _ in range(2**n - 1):
            gate_type = random.choice(['AND', 'OR'])
            inputs = sorted(random.sample(range(n), random.randint(1, n)))
            circuit.append((gate_type, inputs))
        return circuit

    def term_overlap_graph(circuit):
        n = len(circuit[0][1])
        graph = [[0] * (2**n) for _ in range(2**n)]
        for gate in circuit:
            if gate[0] == 'AND':
                for i in gate[1]:
                    for j in gate[1]:
                        if i != j:
                            graph[i][j] += 1
            elif gate[0] == 'OR':
                for i in gate[1]:
                    for j in range(n):
                        if j not in gate[1]:
                            graph[i][j] += 1
        return graph

    def eta_invariant(graph):
        n = len(graph)
```

```

total_edges = sum(sum(row) for row in graph) // 2
max_degree = max(max(row) for row in graph)
return Fraction(total_edges, max_degree)

def monotone_width(circuit):
    n = len(circuit[0][1])
    assignments = [list(range(n)) for _ in range(2**n)]

    for gate in circuit:
        new_assignments = []
        for assignment in assignments:
            if all(assignment[i] == (gate[1][i] % 2) for i in gate[1]):
                new_assignment = list(assignment)
                for i in gate[1]:
                    new_assignment[i] = (new_assignment[i] + 1) % 2
                new_assignments.append(new_assignment)
        assignments = new_assignments

    return len(assignments)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    circuit = generate_circuit(n)
    graph = term_overlap_graph(circuit)
    eta = eta_invariant(graph)
    w_m = monotone_width(circuit)

    results.append({
        "n": n,
        "eta": eta,
        "w_m": w_m
    })

correlation = 0.0
for result in results:
    correlation += (result["eta"] - result["w_m"]) ** 2

correlation /= len(results)
correlation = math.sqrt(correlation)

return {
    "metric_name": "correlation",
    "metric_value": correlation,
    "instances_tested": len(results),
    "n_max": max(result["n"] for result in results),
    "conjecture_holds": correlation > 0.8,
    "counterexample": "" if correlation > 0.8 else "Correlation below threshold"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] + list(range(31))

    results = []

```

```

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {{\"seed\": {seed}, \"metric_name\": \"{trial_result['metric_name']}\", \"m
    results.append(trial_result)

mean_value = sum(result["metric_value"] for result in results) / len(results)
std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) /
support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

if all(result["conjecture_holds"] for result in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
else:
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con
    print(f"RESULT: FALSIFIED counterexample=\"Correlation below threshold\" first_failing_seed

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0d795ae.py", line 104, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0d795ae.py", line 74, in run_trial
    w_m = monotone_width(circuit)
          ^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0d795ae.py", line 58, in monotone_width
    if all(assignment[i] == (gate[1][i] % 2) for i in gate[1]):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f0d795ae.py", line 58, in <genexpr>
    if all(assignment[i] == (gate[1][i] % 2) for i in gate[1]):
           ~~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the conjecture's support or falsification. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17877
2	propose	ollama_remote	glm4:latest	0	0	12874
3	preregistration	ollama_remote	glm4:latest	0	0	9245
4	novelty	ollama_remote	glm4:latest	0	0	8269
5	novelty	ollama_remote	glm4:latest	0	0	8775
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17991
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13946
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13946
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12615
10	judge	ollama_remote	glm4:latest	0	0	12005

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 127543 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/63ead587425e.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/63ead587425e.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/63ead587425e.tar.gz` (if generated)